FIAP ON

SAFRA DO CONHECIMENTO CIBERNÉTICO - PRAGAS VIRTUAIS

COMANDOS DDL NA PRÁTICA - CRIANDO TABELAS COM CREATE TABLE

6

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

LISTA DE FIGURAS

Figura 1 - Oracle Database 8
Figura 2 - SQL Server 8
Figura 3 - MySQL 8
Figura 4 - Constraint Primary Key 13
Figura 5 - Constraint Foreign Key 14
Figura 6 - Constraint NOT NULL/NULL.....15
Figura 7 - Constraint UNIQUE.....15
Figura 8 - Constraint CHECK 16
Figura 9 - Tabela Departamento a partir do Data Modeler, modelo relacional.....19
Figura 10 - Tabela Dependente a partir do Data Modeler, modelo relacional 21
Figura 11 - Tabela Departamento – Constraint Unique, partir do Data Modeler, modelo relacional 22
Figura 12 - Tabela Projeto, partir do Data Modeler, modelo relacional 23
Figura 13 - Tabela Projeto – Constraint CHECK, partir do Data Modeler, modelo relacional.....23
Figura 14 - Relacionamento entre Funcionário e Departamento, partir do Data Modeler, modelo relacional 24
Figura 15 - Exemplo das informações atuais armazenadas em planilhas.....28
Figura 16 - Modelo Relacional (Físico).....32

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

LISTA DE CÓDIGO-FONTE

Código-fonte 1 - Sintaxe comando CREATE.....11
Código-fonte 2 - Sintaxe comando ALTER 11
Código-fonte 3 - Sintaxe comando DROP.....12
Código-fonte 4 - Constraint DEFAULT 17
Código-fonte 5 - Exemplo comando SQL – Create Table 19
Código-fonte 6 - Exemplo comando SQL – Alter Table – Primary Key 20
Código-fonte 7 - Implementação de constraint ao nível de tabela.....21
Código-fonte 8 - Implementação de constraint ao nível de coluna.....21
Código-fonte 9 - Implementação de constraint chave primária composta.....22
Código-fonte 10 - Implementação de constraint UNIQUE.....22
Código-fonte 11 - Implementação de constraint CHECK 23
Código-fonte 12 - Implementação de constraint FOREIGN KEY 24
Código-fonte 13 - Exemplo comando SQL – Drop Table 25
Código-fonte 14 - Exemplo comando SQL – Drop Table com PURGE.....26
Código-fonte 15 - Exemplo comando SQL – Truncate Table.....27
Código-fonte 16 - Exemplo comando SQL – COMMENT 27
Código-fonte 17 - Script de criação do banco de dados.....35

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

LISTA DE QUADROS

Quadro 1 - Instruções da divisão DDL da linguagem SQL 10
Quadro 2 - Prefixos para nomes de constraints 17

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

SUMÁRIO

1 COMANDOS DDL NA PRÁTICA - CRIANDO TABELAS COM CREATE TABLE..6
1.1 Structured Query Language (SQL) ou linguagem estruturada para consulta.....6
1.1.1 Divisões da Linguagem SQL 8
1.1.2 Criando um banco de dados 9
1.1.3 Comandos DDL para criação e manutenção de tabelas 10
1.1.4 Constraints – Regras ou restrições aplicadas à estrutura física de armazenamento 12
1.1.4.1 Constraint Primary Key.....13
1.1.4.2 Constraint Foreign Key.....13
1.1.4.3 Constraint NOT NULL 14
1.1.4.4 Constraint UNIQUE 15
1.1.4.5 Constraint CHECK 16
1.1.4.6 Constraint DEFAULT.....16
1.1.4.7 Recomendações para adição de constraints.....17
1.1.5 Transformação do modelo relacional para implementação através de comandos da linguagem SQL 18
1.1.5.1 Recomendação para implementação de um modelo relacional para comandos SQL.....18
1.1.5.2 Implementação de uma tabela utilizando o comando CREATE TABLE.....18
1.1.5.3 Implementação de CONSTRAINTS utilizando o comando ALTER TABLE...19
1.1.5.3.1 Implementação da constraint Primary Key 20
1.1.5.3.2 Implementação da constraint Unique 22
1.1.5.3.3 Implementação da constraint Check 23
1.1.5.3.4 Implementação da constraint Foreign Key 24
1.1.5.4 Implementação da exclusão das tabelas, utilizando o comando DROP TABLE.....24
1.1.5.5 Comentários associados a tabelas e campos através da linguagem SQL 27
1.1.6 Hands On – Transformação do modelo relacional para implementação através de comandos da linguagem SQL 28
1.1.6.1 Estudo de Caso: SIP – SISTEMA DE CONTROLE DE IMPLANTACAO DE PROJETOS 28
1.1.6.2 Proposta de solução para a representação do modelo relacional (físico) de dados a partir da ferramenta Oracle Data Modeler 32
1.1.6.3 Comandos da divisão DDL, da linguagem SQL para implementar o banco de dados referente ao estudo de caso SIP 32
REFERÊNCIAS.....36

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

1 COMANDOS DDL NA PRÁTICA - CRIANDO TABELAS COM CREATE TABLE

1.1 Structured Query Language (SQL) ou linguagem estruturada para consulta

Structured Query Language (SQL) ou linguagem estruturada para consulta é uma linguagem de definição, manipulação e controle de banco de dados. Ela representa um conjunto de comandos responsáveis pela definição das tabelas, seleção e atualização dos dados em um SGBD.

A linguagem SQL é totalmente padronizada, reconhecida pelo ANSI (American National Standard Institute – Instituto Nacional Americano de Padrões) e possui uma sintaxe simples, facilitando o aprendizado e o uso dessa linguagem.

Trazendo um pouco da história, podemos ilustrar por meio de uma linha de tempo:

- **Início de 1970:** o departamento de pesquisas da IBM desenvolveu a linguagem SQL, como forma de interface para o sistema de banco de dados relacional (SGBDR) denominado System R. Inicialmente a linguagem SQL foi chamada de SEQUEL (Structured English Query Language).
- **1977:** a nomenclatura foi revisada e passou a ser chamada definitivamente de SQL.
- **1986:** ANSI, em conjunto com a ISO (International Standards Organization – Organização Internacional de Padrões), publicou o padrão de linguagem SQL, conhecido como SQL-86 ou SQL-1, tornando-se a linguagem padrão adotada para os bancos de dados relacionais.

O primeiro SGBD lançado no mercado, implementando a linguagem SQL, foi o Oracle (que era chamado de Relational Software) em 1979.

Embora a linguagem SQL seja um padrão para bancos relacionais, há diferenças nas implementações dessa linguagem nos vários SGBDRs de mercado.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Cada fabricante de software cria extensões (com sintaxes, funções, recursos etc.) específicas de seus sistemas, formando diferentes “dialetos” e “versões” da linguagem.

É possível escrever aplicações utilizando apenas o SQL padrão ANSI (com os comandos comuns à maioria dos SGBDRs).

Porém, na prática, muitas vezes é necessário o uso das sintaxes proprietárias para aproveitar recursos específicos ou conseguir melhor desempenho.

Com a padronização, a linguagem SQL apresenta algumas características que destacamos a seguir:

- As instruções padronizadas seguem a mesma nomenclatura e formato para os diferentes tipos de SGBD, respeitando as particularidades de cada um.
- A migração de um SGBDR para outro não requer grandes mudanças.
- Quando ocorre a migração de um SGBDR, a adaptação dos profissionais é facilitada, com redução de tempo e de custos para os treinamentos, pois as instruções possuem nomes e funcionalidades iguais ou muito semelhantes.
- Portabilidade entre plataformas.
- As queries (instruções) do SQL têm uma estrutura semelhante à linguagem natural (em inglês).
- As instruções de manipulação de dados sempre trabalham com tabelas e/ou retornam resultados em forma de tabelas (com linhas e colunas).
- É uma linguagem não procedural, isto é, as sentenças em SQL declaram a tarefa a ser realizada (“o que”) sem a necessidade de especificar os procedimentos passo a passo que o sistema deve seguir para executá-la (“como”).
- Independência física dos dados e dispositivos não especifica o método de acesso aos dados e nem como estão armazenados (os SGBDRs possuem um “otimizador” que se encarrega da maneira de recuperar os dados).

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Comandos DDL na prática - Criando tabelas com Create Table

Para ilustrar, vejamos um exemplo apresentando algumas diferenças de sintaxe entre os SGBDRs, para recuperar a primeira linha de uma tabela.

```
SELECT * FROM T_CLIENTE WHERE ROWNUM = 1;
```

Figura 1 - Oracle Database
Fonte: Google (2022)

```
SELECT TOP 1 * FROM T_CLIENTE;
```

Figura 2 - SQL Server
Fonte: Google (2022)

```
SELECT * FROM T_CLIENTE LIMIT 1;
```

Figura 3 - MySQL
Fonte: Google (2022)

1.1.1 Divisões da Linguagem SQL

- **DDL (Data Definition Language) – Linguagem de Definição de Dados:** utilizada para definir tabelas e elementos associados à estrutura física do banco de dados.

  ○ Exemplos: CREATE, DROP, ALTER, RENAME, TRUNCATE.

- **DML (Data Manipulation Language) – Linguagem de Manipulação de Dados:** utilizada para inserir, atualizar e apagar dados, ou seja, para modificar o conteúdo das tabelas.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

  - Exemplos: INSERT, UPDATE, DELETE.

- **DCL (Data Control Language) – Linguagem de Controle de Dados:** utilizada para controlar autorização de acesso a dados e operações.

  - Exemplos: GRANT, REVOKE.

- **DTL (Data Transaction Language) – Linguagem de Transação de Dados:** utilizada para controlar as transações de banco de dados, permitindo o registro permanente das alterações ou desfazendo as alterações.

  - Exemplos: COMMIT, ROLLBACK, SAVEPOINT.

- **DQL (Data Query Language) ou DRS (Data Retrieve Language) – Linguagem de Consulta de Dados:** utilizada para especificar consultas; é composta por várias cláusulas e opções, o que possibilita a elaboração de consultas simples a complexas.

  - Exemplo: SELECT.

Alguns SGBDRs que utilizam a linguagem SQL: Apache Derby, Caché, DB2, Firebird, Informix, Interbase, SQL Server, MySQL, Oracle, PostGreSQL, Sybase.

ANSI e ISO são responsáveis pela padronização, porém há muitas variações criadas pelos fabricantes de SGBD.

1.1.2 Criando um banco de dados

Um banco de dados é implementado após a representação do modelo relacional (físico) de dados e aplicação da normalização/desnormalização de dados.

Utilizaremos os comandos DDL da linguagem SQL para implementar as tabelas de um banco de dados.

É importante salientar que os comandos da linguagem SQL não são sensíveis a letras maiúsculas e minúsculas. O conteúdo armazenado é normalmente sensível a

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

letras maiúsculas e minúsculas, portanto orienta-se atenção durante as comparações de conteúdos alfanuméricos.

Os SGBDRs utilizam terminadores nas suas linhas de comando. No caso do Oracle, o terminador utilizado é o ponto e vírgula (;). Sendo assim, é importante verificar o terminador utilizado pelo SGBDR escolhido para implementação, com essa prática, os comandos serão executados corretamente.

1.1.3 Comandos DDL para criação e manutenção de tabelas

A divisão DDL (Data Definition Language) da linguagem SQL permite a definição das estruturas físicas do banco de dados e a criação do banco de dados, tabelas, campos. Além disso, possibilita as relações entre as tabelas e demais objetos de banco de dados.

São instruções da divisão DDL:

INSTRUÇÃO	DESCRIÇÃO
CREATE	Criação das estruturas físicas do banco de dados
ALTER	Alteração das estruturas físicas do banco de dados
DROP	Eliminação das estruturas físicas do banco de dados
TRUNCATE	Exclusão física das linhas da tabela
RENAME	Renomeação das estruturas físicas do banco de dados
COMMENT	Inserção de comentários relativos as estruturas físicas do banco de dados

Quadro 1 - Instruções da divisão DDL da linguagem SQL
Fonte: Elaborado pelo autor (2022)

Comando CREATE TABLE:

Sintaxe para criar a estrutura de uma tabela definindo as colunas (campos) e as chaves primárias e estrangeiras existentes (relacionamento PK vs FK).

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
-- Sintaxe:

CREATE TABLE <nome-tabela> (

<nome-coluna> <tipo-do-dado> [NOT NULL]

PRIMARY KEY (nome-coluna-chave)

FOREIGN KEY (nome-coluna-chave-estrangeira)

REFERENCES <nome-tabela-pai> (nome-coluna-chave-primária));
```

Código-fonte 1 - Sintaxe comando CREATE
Fonte: Elaborado pelo autor (2022)

Comando ALTER TABLE:

Altera a estrutura de uma tabela acrescentando, alterando, retirando e alterando nomes, formatos das colunas e a integridade referencial definidas em uma determinada tabela.

```
-- Sintaxe:

ALTER TABLE <nome-tabela>

DROP COLUMN <nome-coluna>

ADD  <nome-coluna> <tipo-do-dado> [NOT NULL]

[NOT NULL WITH DEFAULT]

RENAME <nome-coluna> <novo-nome-coluna>

MODIFY <nome-coluna> <tipo-do-dado> [NULL]

[NOT NULL]
```

Código-fonte 2 - Sintaxe comando ALTER
Fonte: Elaborado pelo autor (2022)

Comando DROP TABLE:

Deleta (apaga) a estrutura e os dados existentes em uma tabela. Após a execução deste comando, estarão deletados todos os dados, estrutura e índices de acessos que estejam a ela associados.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
-- Sintaxe:

DROP TABLE <nome-tabela>;
```

Código-fonte 3 - Sintaxe comando DROP
Fonte: Elaborado pelo autor (2022)

1.1.4 Constraints – Regras ou restrições aplicadas à estrutura física de armazenamento

As constraints são parte opcional da cláusula CREATE TABLE e ALTER TABLE. São regras agregadas aos campos (colunas) e tabelas.

As restrições (constraints) evitam:

- Impedir que dados inválidos sejam cadastrados no banco de dados.
- Garantir a qualidade dos dados.
- Garantir que campos (atributos) obrigatórios sejam preenchidos.
- Garantir que uma tabela (entidade) não seja deletada se houver dependências.
- Garantir a integridade referencial definida no modelo relacional.
- Em geral, fazem parte da especificação do projeto conceitual do banco de dados e estão descritas no modelo relacional.

Tipos de Constraints no Oracle Database:

- PK – PRIMARY KEY
- FK – FOREIGN KEY
- NOT NULL/ NULL
- UN – UNIQUE CONSTRAINT
- CK – CHECK CONSTRAINT
- DEFAULT – Valor padrão

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

1.1.4.1 Constraint Primary Key

A Chave Primária (ou Primary Key) é a restrição/regra que identifica cada linha (registro/ocorrência/instância/tupla) da tabela (entidade) de forma única (garante a unicidade).

Ela promove a unicidade das linhas, proporcionando uma maneira de identificar de forma única cada item que é armazenado em uma tabela. Além disso, pode ser composta por um único campo (coluna) ou por um conjunto de campos (chave primária composta).

É importante destacar que os campos (colunas) que formam uma PK são obrigatórios, ou seja, não podem aceitar valores nulos. E também que os valores de uma chave primária não se repetem.

Representação da definição da chave primária através da ferramenta Oracle Data Modeler – Modelo Relacional.

Observação: a chave primária é representada pela letra P (Primary Key).

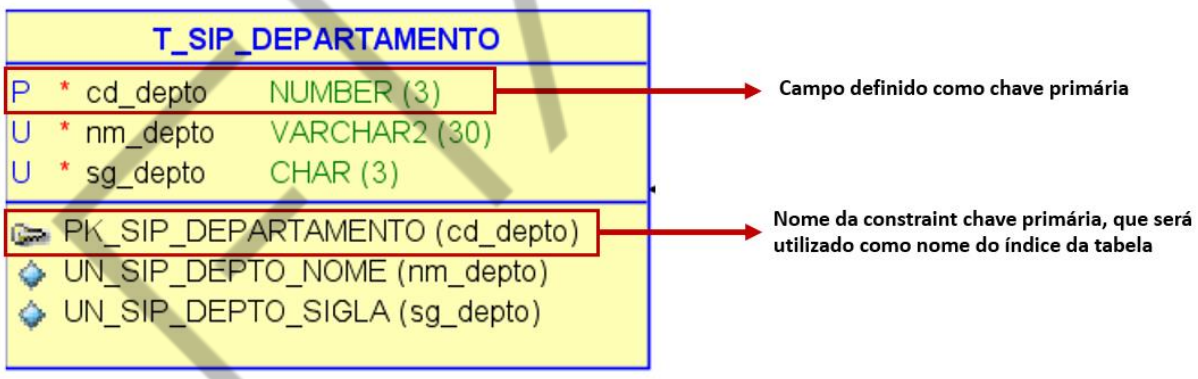

Figura 4 - Constraint Primary Key
Fonte: Elaborado pelo autor (2022)

1.1.4.2 Constraint Foreign Key

A Chave Estrangeira (Foreign Key) é a restrição/regra que estabelece a relação-relacionamento (associação) entre duas tabelas (entidades).

Ela garante a integridade referencial, ou seja, toda chave estrangeira faz referência a uma chave primária de uma tabela referenciada.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Os relacionamentos são analisados aos pares de tabelas (entidades), portanto a tabela (entidade) que recebe a chave estrangeira é a tabela (entidade) que possui a cardinalidade máxima do relacionamento igual a 1 (monovalorada). Em outras palavras, a chave estrangeira é um campo (atributo) monovalorado, isto é, possui apenas um único valor.

Representação da definição da chave estrangeira através da ferramenta Oracle Data Modeler – Modelo Relacional.

Observação: a chave estrangeira é indicada com a letra F (Foreign Key).

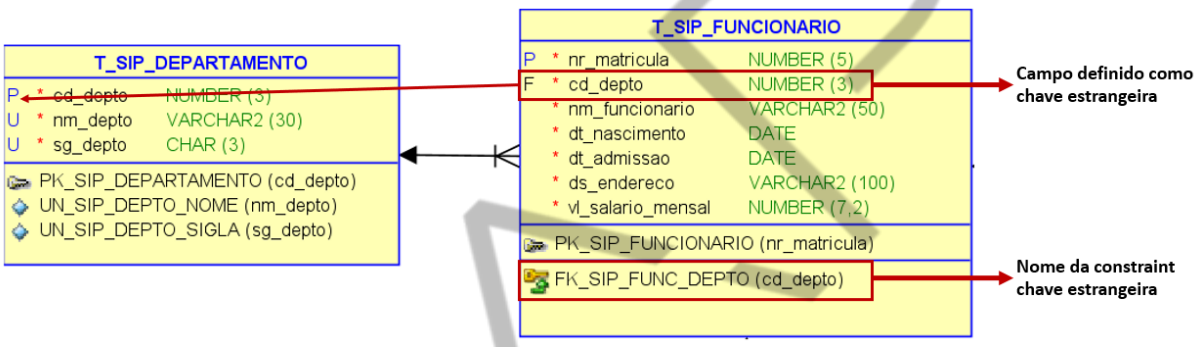

Figura 5 - Constraint Foreign Key
Fonte: Elaborado pelo autor (2022)

1.1.4.3 Constraint NOT NULL

A Constraint NOT NULL é a restrição/regra que indica um tipo de checagem (check constraint) utilizada para definir campos (atributos) obrigatórios (mandatórios).

É importante ressaltar que um campo (atributo) obrigatório não pode aceitar valor nulo — nulo indica ausência de valor (não definido ou desconhecido) para um campo (atributo).

Representação da definição da coluna obrigatória/opcional através da ferramenta Oracle Data Modeler – Modelo Relacional.

Observação: o not null é indicado com o * (asterisco), já que é um campo opcional, não possui nenhuma indicação (ausência da obrigatoriedade).

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

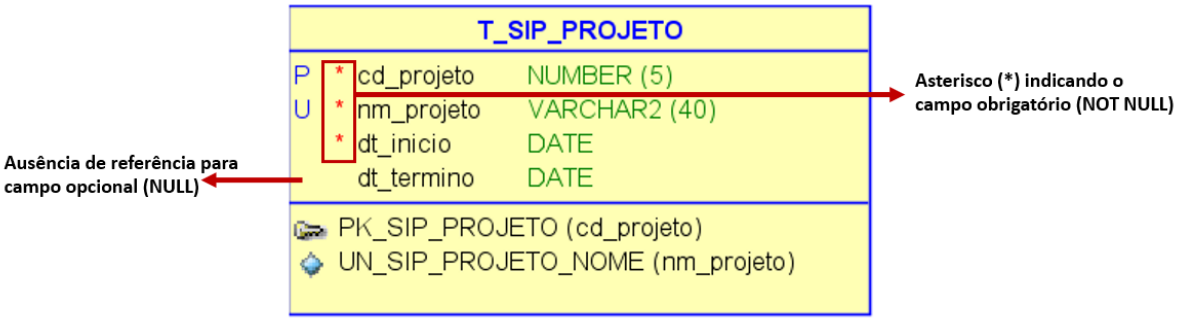

Figura 6 - Constraint NOT NULL/NULL
Fonte: Elaborado pelo autor (2022)

1.1.4.4 Constraint UNIQUE

UNIQUE CONSTRAINT é a restrição/regra que indica que uma determinada coluna (atributo/campo) de uma tabela não poderá ter repetição, ou seja, não poderá ter valores repetidos para o conteúdo da coluna.

Quando indicamos que uma coluna deve conter valores únicos (sem repetição), indicamos que todos os valores não nulos devem ser exclusivos.

Uma coluna definida como Unique Constraint pode ser uma coluna opcional (aceita valores nulos – ausência de valores). É importante ressaltar que em função da Constraint Unique não permitir repetição de valores, apenas uma linha (registro) da tabela poderá ter a coluna com valor nulo (ausência de valor).

Representação da definição Constraint Unique através da ferramenta Oracle Data Modeler – Modelo Relacional.

Observação: o campo definido com a Constraint Unique é indicado com a letra U (UNIQUE) na frente do campo.

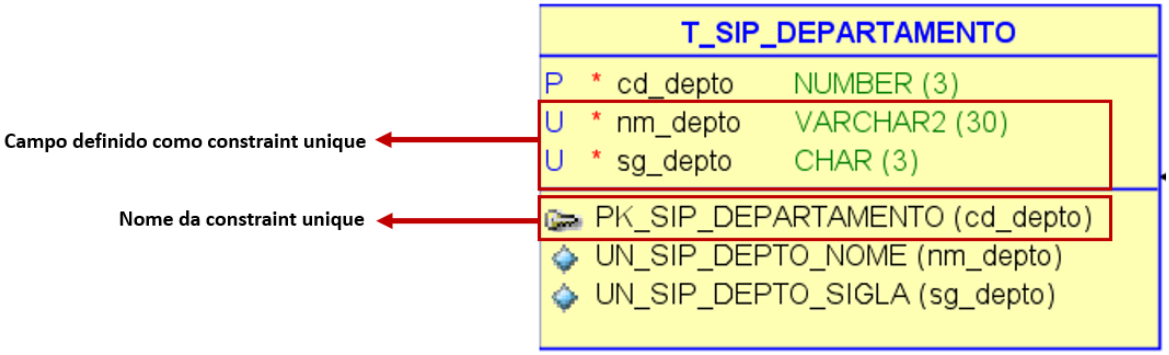

Figura 7 - Constraint UNIQUE
Fonte: Elaborado pelo autor (2022)

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

1.1.4.5 Constraint CHECK

A Constraint CHECK é a restrição/regra que define o domínio de um determinado campo (atributo). É uma constraint de verificação ou validação. O domínio é uma expressão que se refere aos valores possíveis para o conteúdo de um campo.

É utilizada uma verificação para garantir a integridade de dados conforme uma regra de negócio estabelecida.

Utiliza expressões lógicas (semelhante ao comando “IF” das linguagens de programação) para definir a validação necessária.

Representação da definição constraint check através da ferramenta Oracle Data Modeler – Modelo Relacional.

O campo definido com a constraint check é configurado através da edição da tabela, através do data modeler. A constraint check não é apresentada de forma explicita no modelo relacional.

	Nome	Regra de Validação
1	CK_SIP_FUNC_SALARIO	VL_SALARIO_MENSAL >= 1212

Figura 8 - Constraint CHECK
Fonte: Elaborado pelo autor (2022)

1.1.4.6 Constraint DEFAULT

A constraint DEFAULT atribui um conteúdo padrão a uma coluna da tabela sempre que for incluída uma nova linha na tabela.

É normalmente definida durante a implementação do banco de dados, através dos comandos SQL. No momento da criação da tabela, define-se o respectivo campo, que precisa ter um valor default, conforme regras de negócio.

Considerando uma regra de negócio para exemplificar, que define que a quantidade vendida de um produto deve corresponder ao menos uma unidade para cada venda realizada. Podemos definir a constraint default através da linguagem SQL, comando DDL, conforme fragmento de código a seguir.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
-- Sintaxe:

.

.

.

    qt_vendida NUMBER(4) DEFAULT 1 ,

.

.

.
```

Código-fonte 4 - Constraint DEFAULT
Fonte: Elaborado pelo autor (2022)

1.1.4.7 Recomendações para adição de constraints

Normalmente, utilizamos o comando ALTER TABLE para inserir restrições (Constraints).

É uma boa prática adicionar ou eliminar restrições, mas não modificar restrições.

Adicione uma restrição NOT NULL utilizando a cláusula MODIFY.

É recomendado que se especifique sempre um nome significativo para as constraints (restrições). Dessa forma, quando ocorrer um erro relacionado às constraints (restrições), ficará mais fácil de identificar o problema ou mesmo sua manutenção quando necessário.

Da mesma forma que nomeamos tabelas e campos, precisamos nomear as constraints.

Sugerimos a seguir uma lista de prefixos para serem utilizados nos nomes das constraints, facilitando o entendimento e posterior manutenção.

Prefixo para constraints	Descrição
PK	Constraint Primary Key
FK	Constraint Foreign Key
UN	Constraint Unique
CK	Constraint Check

Quadro 2 - Prefixos para nomes de constraints
Fonte: Elaborado pelo autor (2022)

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

1.1.5 Transformação do modelo relacional para implementação através de comandos da linguagem SQL

1.1.5.1 Recomendação para implementação de um modelo relacional para comandos SQL

Durante a transformação do modelo relacional para comandos SQL, entendemos como boa prática a implementação sendo feita da seguinte maneira:

- Criar as tabelas contendo campos, tipos de dados e tamanho, utilizando o comando CREATE TABLE.
- Criar as chaves primárias e demais constraints (exceto chaves estrangeiras) utilizando o comando ALTER TABLE.
- Criar as chaves estrangeiras utilizando o comando ALTER TABLE, após todas as tabelas, respectivas chaves primárias e demais constraints.

A vantagem da escrita ser realizada desta maneira é que o código fica mais simples de ser lido e mantido.

Ao criar uma tabela utilizando o comando CREATE TABLE com chaves primárias, estrangeiras e demais constraints, temos como desvantagem a ordem de criação das tabelas, conforme dependência das tabelas e os seus relacionamentos. Dessa forma, o comando se torna mais complexo e mais difícil de ser mantido. Caso necessite de alguma alteração, será necessária a reescrita de todo o bloco de comandos.

1.1.5.2 Implementação de uma tabela utilizando o comando CREATE TABLE

Quando implementamos uma tabela, é fundamental fazer isso a partir do modelo relacional de dados, que reflete o projeto físico do banco de dados.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

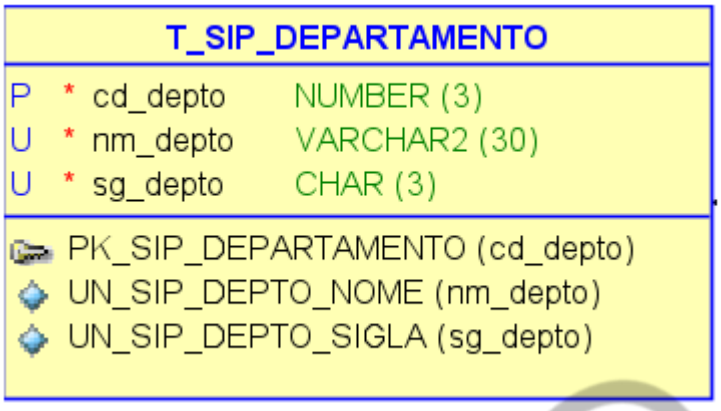

Figura 9 - Tabela Departamento a partir do Data Modeler, modelo relacional
Fonte: Elaborado pelo autor (2022)

A partir da tabela ilustrada na figura anterior, através do modelo relacional, podemos criar o comando na linguagem SQL, responsável pela implementação da tabela no SGDBR Oracle.

A seguir, será apresentado o código na linguagem SQL, que permite criar a tabela a partir da sintaxe geral que foi apresentada, e seguindo as recomendações para implementação do código.

```
-- CRIAÇÃO DA TABELA

CREATE TABLE t_sip_departamento (

    cd_depto NUMBER(3)        NOT NULL,

    nm_depto VARCHAR2(30)     NOT NULL,

    sg_depto CHAR(3)          NOT NULL

);
```

Código-fonte 5 - Exemplo comando SQL – Create Table
Fonte: Elaborado pelo autor (2022)

1.1.5.3 Implementação de CONSTRAINTS utilizando o comando ALTER TABLE

Normalmente utilizamos o comando ALTER TABLE para inserir restrições (Constraints).

É uma boa prática adicionar ou eliminar restrições, mas não modificar essas restrições.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

É recomendado que se especifique sempre um nome significativo para as restrições (constraints). Dessa forma, quando ocorrer um erro de restrições, ficará mais fácil identificar o problema.

1.1.5.3.1 Implementação da constraint Primary Key

Com base na tabela da figura “Tabela Departamento a partir do Data Modeler, modelo relacional”, iremos implementar a constraint Primary Key.

A seguir, apresentamos o código na linguagem SQL, que permite adicionar a constraint Primary Key à tabela, a partir da sintaxe geral que foi apresentada e seguindo as recomendações para implementação do código.

```
-- IMPLEMENTAÇÃO DA PRIMARY KEY COM ALTER TABLE

   ALTER TABLE t_sip_departamento

ADD CONSTRAINT pk_sip_departamento

   PRIMARY KEY ( cd_depto );
```

Código-fonte 6 - Exemplo comando SQL – Alter Table – Primary Key
Fonte: Elaborado pelo autor (2022)

Podemos adicionar constraints em uma tabela de duas formas:

1. Como parte do comando CREATE TABLE, ou seja, quando a tabela é criada. Nesse caso a constraint pode ser criada de duas formas:

   a) ao nível de coluna; ou

   b) ao nível de tabela.

2. Como parte do comando ALTER TABLE, ou seja, após a criação da tabela. Essa e a forma mais comum de implementar as constraints.

No exemplo do código-fonte “Exemplo comando SQL – Alter Table – Primary Key”, implementamos a chave primária através do comando ALTER TABLE.

A seguir, exemplificamos a implementação feita a partir do comando CREATE TABLE tanto ao nível de coluna como ao nível de tabela.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
-- CONSTRAINT PRIMARY KEY A NÍVEL DE TABELA

CREATE TABLE t_sip_departamento (

    cd_depto NUMBER(3)       NOT NULL,

    nm_depto VARCHAR2(30)    NOT NULL,

    sg_depto CHAR(3)         NOT NULL,

CONSTRAINT pk_sip_departamento PRIMARY KEY ( cd_depto )

);
```

Código-fonte 7 - Implementação de constraint ao nível de tabela
Fonte: Elaborado pelo autor (2022)

```
-- CONSTRAINT PRIMARY KEY A NÍVEL DE COLUNA

CREATE TABLE t_sip_departamento (

    cd_depto NUMBER(3)       NOT NULL

                  CONSTRAINT pk_sip_departamento PRIMARY KEY,

    nm_depto VARCHAR2(30)    NOT NULL,

    sg_depto CHAR(3)         NOT NULL

);
```

Código-fonte 8 - Implementação de constraint ao nível de coluna
Fonte: Elaborado pelo autor (2022)

Podemos exemplificar a implementação de uma **chave primária composta** por meio do comando ALTER TABLE.

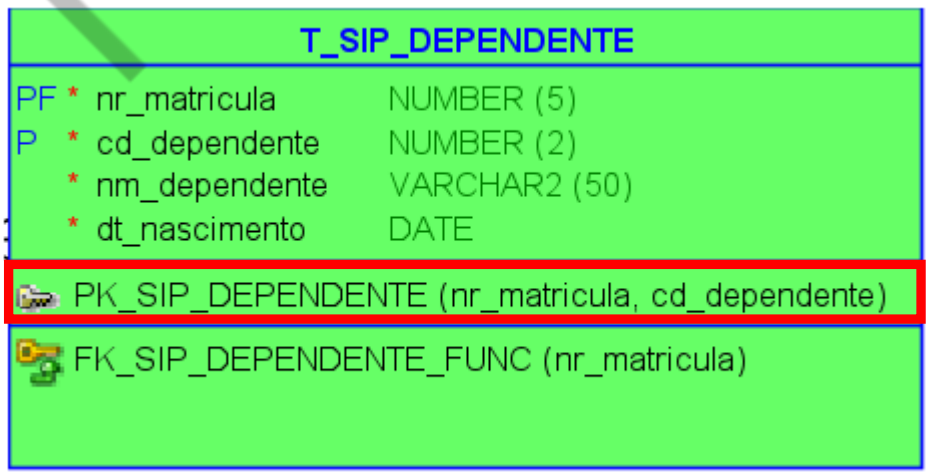

Figura 10 - Tabela Dependente a partir do Data Modeler, modelo relacional
Fonte: Elaborado pelo autor (2022)

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Considerando a tabela anterior, através do modelo relacional, podemos criar o comando na linguagem SQL, responsável pela implementação da constraint Primary Key composta por dois campos.

```
-- CONSTRAINT PRIMARY KEY COMPOSTA POR DOIS CAMPOS

   ALTER TABLE t_sip_dependente

ADD CONSTRAINT pk_sip_dependente

   PRIMARY KEY ( nr_matricula, cd_dependente );
```

Código-fonte 9 - Implementação de constraint chave primária composta
Fonte: Elaborado pelo autor (2022)

1.1.5.3.2 Implementação da constraint Unique

Com base na tabela da figura “Tabela Departamento – Constraint Unique, partir do Data Modeler, modelo relacional”, implementaremos a constraint Unique.

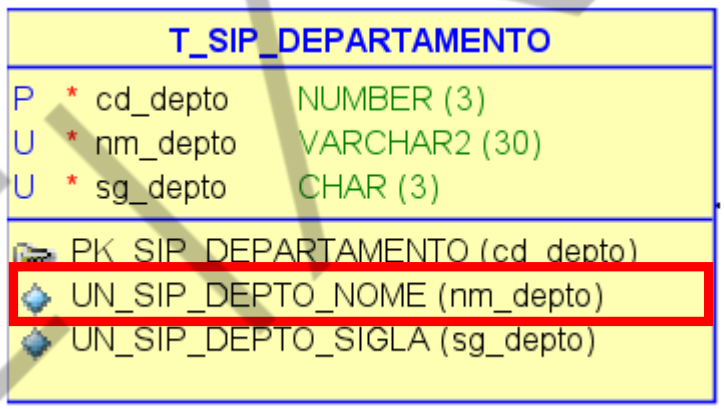

Figura 11 - Tabela Departamento – Constraint Unique, partir do Data Modeler, modelo relacional
Fonte: Elaborado pelo autor (2022)

Apresentamos o código na linguagem SQL, que permite adicionar a constraint Unique à tabela, a partir da sintaxe geral que foi apresentada e seguindo as recomendações para implementação do código.

```
-- IMPLEMENTAÇÃO DA CONSTRAINT UNIQUE

   ALTER TABLE t_sip_departamento

ADD CONSTRAINT un_sip_depto_nome

         UNIQUE ( nm_depto );
```

Código-fonte 10 - Implementação de constraint UNIQUE
Fonte: Elaborado pelo autor (2022)

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

1.1.5.3.3 Implementação da constraint Check

Com base na tabela da figura “Tabela Projeto, partir do Data Modeler, modelo relacional”, implementaremos a constraint Check.

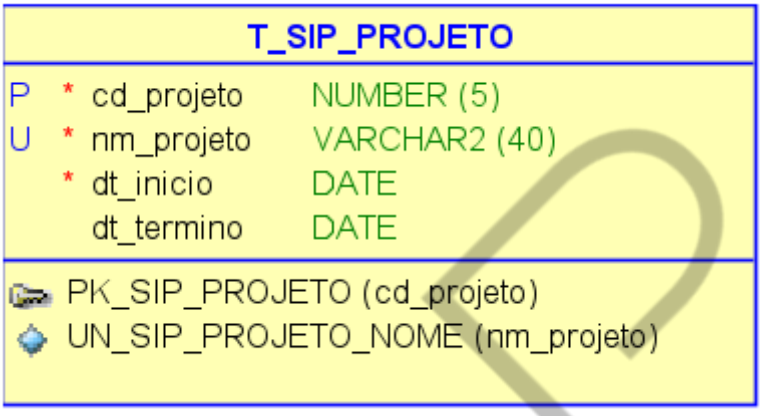

Figura 12 - Tabela Projeto, partir do Data Modeler, modelo relacional
Fonte: Elaborado pelo autor (2022)

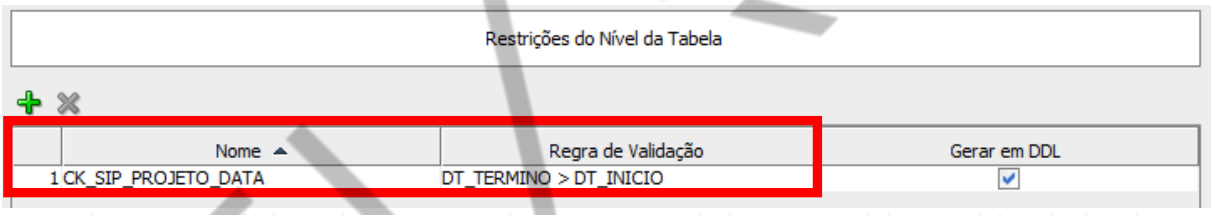

Figura 13 - Tabela Projeto – Constraint CHECK, partir do Data Modeler, modelo relacional
Fonte: Elaborado pelo autor (2022)

Apresentamos o código na linguagem SQL, que permite adicionar a constraint Check à tabela através da sintaxe geral que foi apresentada e seguindo as recomendações para implementação do código.

```
-- IMPLEMENTAÇÃO DA CONSTRAINT CHECK

   ALTER TABLE t_sip_projeto

ADD CONSTRAINT ck_sip_projeto_data

          CHECK ( dt_termino > dt_inicio );
```

Código-fonte 11 - Implementação de constraint CHECK
Fonte: Elaborado pelo autor (2022)

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

1.1.5.3.4 Implementação da constraint Foreign Key

Com base na tabela ilustrada pela figura “Relacionamento entre Funcionário e Departamento, partir do Data Modeler, modelo relacional”, implementaremos a constraint Foreign Key.

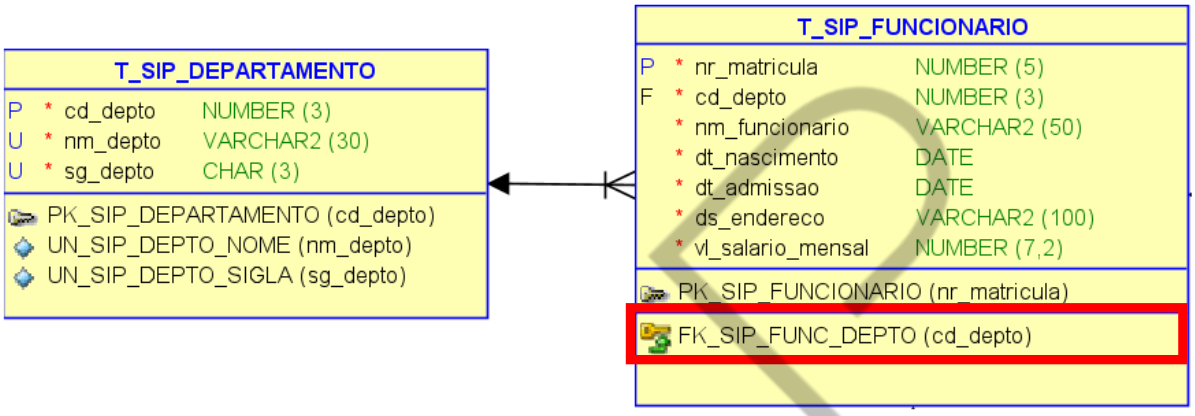

Figura 14 - Relacionamento entre Funcionário e Departamento, partir do Data Modeler, modelo relacional
Fonte: Elaborado pelo autor (2022)

Apresentamos o código na linguagem SQL, que permite adicionar a constraint Foreign Key à tabela, a partir da sintaxe geral que foi apresentada e seguindo as recomendações para implementação do código.

```
-- IMPLEMENTAÇÃO DA CONSTRAINT FOREIGN KEY

   ALTER TABLE t_sip_funcionario

ADD CONSTRAINT fk_sip_func_depto

   FOREIGN KEY ( cd_depto )

    REFERENCES t_sip_departamento ( cd_depto );
```

Código-fonte 12 - Implementação de constraint FOREIGN KEY
Fonte: Elaborado pelo autor (2022)

1.1.5.4 Implementação da exclusão das tabelas, utilizando o comando DROP TABLE

O comando DROP TABLE exclui uma tabela permanentemente, incluindo seus dados e índices.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

A partir da versão 10g do Oracle Database, quando uma tabela é afetada pelo comando DROP TABLE, o SGBDR não remove definitivamente a tabela, mas a envia para a lixeira, conhecida como RECYCLE BIN. Dessa forma, é possível recuperar a tabela excluída (esta funcionalidade do Oracle é denominada Flashback Drop e a recuperação é feita através do comando Flashback Table).

Para facilitar a manutenção do banco de dados, podemos deixar os comandos para exclusão previamente definidos, caso seja necessário destruir a estrutura de dados em algum momento do desenvolvimento.

Inserimos as instruções a seguir no início do arquivo de comandos SQL. Normalmente deixamos esta estrutura comentada, em função de não ser executada com frequência, a fim de facilitar seu uso quando necessário.

Apresentamos o código na linguagem SQL, que permite excluir a tabela a partir da sintaxe geral que foi apresentada e seguindo as recomendações para implementação do código.

```
-- EXCLUIR A TABELA

DROP TABLE t_sip_departamento;
```

Código-fonte 13 - Exemplo comando SQL – Drop Table
Fonte: Elaborado pelo autor (2022)

No momento da exclusão de tabelas, precisamos considerar que existem os relacionamentos que criam dependências entre as tabelas.

Os relacionamentos visam a integridade referencial, ou seja, garantir a referência que uma tabela faz a outra. Exemplificando: garantir que um funcionário trabalhe em um departamento que esteja cadastrado.

Quando excluímos uma tabela, o SGBDR avalia se existe alguma referência a ela, e caso exista, ele não permitirá a exclusão, garantindo a integridade referencial. Portanto, quando excluímos, precisamos respeitar essas referências.

Para auxiliar a exclusão, é possível excluir uma tabela e eliminar as referências por meio da cláusula CASCADE CONSTRAINTS.

Apresentamos a seguir o código na linguagem SQL, que permite excluir a tabela e eliminar as referências.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
-- EXCLUIR A TABELA ELIMINANDO AS REFERÊNCIAS

DROP TABLE t_sip_departamento CASCADE CONSTRAINTS;
```

Código-fonte 14 - Exemplo comando SQL – Drop Table com Cascade Constraints
Fonte: Elaborado pelo autor (2022)

A cláusula PURGE (a partir da versão 10g do Oracle Database), no final da instrução, permite que a tabela seja eliminada definitivamente, sem que fique na "lixeira" (**RECYCLE BIN**):

Apresentamos o código "Exemplo comando SQL – Drop Table com PURGE" na linguagem SQL, que permite excluir a tabela, utilizando a cláusula PURGE.

```
-- EXCLUIR A TABELA COM PURGE

DROP TABLE t_sip_departamento PURGE;


-- EXCLUIR A TABELA COM PURGE, ELIMINANDO AS REFERÊNCIAS

DROP TABLE t_sip_departamento CASCADE CONSTRAINTS PURGE;
```

Código-fonte 14 - Exemplo comando SQL – Drop Table com PURGE
Fonte: Elaborado pelo autor (2022)

Existe um comando denominado TRUNCATE TABLE. E qual é a diferença entre DROP TABLE vs TRUNCATE TABLE?

O DROP TABLE exclui a tabela de forma permanente, incluindo seus dados e índices. E o TRUNCATE TABLE elimina todas as linhas da tabela, sem excluir a estrutura dos campos.

O comando TRUNCATE TABLE é um comando da divisão DDL, da linguagem SQL, e não permite escolher quais dados serão eliminados. Este comando "limpa/reinicia" a tabela para uma próxima utilização.

É muito mais rápido remover todas as linhas da tabela usando TRUNCATE TABLE do que usando DELETE (divisão DML, da linguagem SQL), que permite escolher quais dados armazenados serão eliminados.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Observe o código a seguir na linguagem SQL que permite excluir todos os dados da tabela, sem eliminar a sua estrutura.

```
-- EXCLUIR TODOS OS DADOS DA TABELA, SEM ELIMINAR A ESTRUTURA

TRUNCATE TABLE < nome-tabela >;
```

Código-fonte 15 - Exemplo comando SQL – Truncate Table
Fonte: Elaborado pelo autor (2022)

1.1.5.5 Comentários associados a tabelas e campos através da linguagem SQL

O comando COMMENT permite adicionar comentários sobre tabelas e colunas.

Os comentários são armazenados no **dicionário de dados** (é uma coleção de metadados que contém definições e representações de elementos de dados) do SGBD, sendo muito útil para enriquecer os descritivos dos **metadadados** (informações sobre os dados).

A seguir, apresentamos o código na linguagem SQL que permite inserir comentários sobre a tabela e campos.

```
-- COMENTARIO SOBRE TABELA E CAMPOS

COMMENT ON TABLE T_SIP_DEPARTAMENTO IS 'EXEMPLO DE COMENTÁRIO
PARA A TABELA';


COMMENT ON COLUMN T_SIP_DEPARTAMENTO.SG_DEPTO IS 'EXEMPLO DE
COMENTÁRIO DE COLUNA';
```

Código-fonte 16 - Exemplo comando SQL – COMMENT
Fonte: Elaborado pelo autor (2022)

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

1.1.6 Hands On – Transformação do modelo relacional para implementação através de comandos da linguagem SQL

1.1.6.1 Estudo de Caso: SIP – SISTEMA DE CONTROLE DE IMPLANTACAO DE PROJETOS

A empresa “TUDO Implanta” precisa melhorar seus controles que hoje se encontram em planilhas.

Para realizar essa tarefa, ela contratou você para traduzir as informações contidas nas planilhas em banco de dados relacional.

Para iniciarmos o entendimento, recebemos a imagem da figura “Exemplo das informações atuais armazenadas em planilhas”, que representa um relatório, contendo basicamente informações de funcionários, dependentes e os projetos.

FUNCIONÁRIO	MATRÍCULA	DATA NASC.	DATA ADMISSÃO	DEPARTAMENTO	SALÁRIO	ENDEREÇO
Ana Maria	1234	10/02/1986	09/08/2010	01 – Financeiro	R$ 1.234,56	R. Dario Pereira, 23
Rosa da Silva	1235	15/08/1970	07/06/2005	02 – Marketing	R$ 2.345,00	Trav. Das Lágrimas, 34
Antonia Camargo	3245	18/07/1986	11/02/2003	01 – Financeiro	R$ 7.654,99	Pç. XV de novembro, 219
João da Silva	2233		08/09/2011	03 – Recursos Humanos	R$ 3.452,12	Av. Nações Unidas, 10200
Joaquim Xavier	7866	21/04/1987	05/04/2010	03 – Recursos Humanos	R$ 980,00	R. Gil Vicente, 45
José Maria	9876	08/09/1998	17/03/2009	04 – Comercial	R$ 1.780,00	Av. Paulista, 23

FUNCIONÁRIO	DEPENDENTE	DATA NASC.
Rosa da Silva	Pedro da Silva	12/01/1963
Antonia Camargo	George Camargo	15/09/1971
Antonia Camargo	Antonio Camargo	10/06/2000
João da Silva	Ana da Silva	23/04/1976
João da Silva	Joana da Silva	16/05/2002
João da Silva	João da Silva Jr.	16/05/2002

PROJETO	DATA INÍCIO	DATA TÉRMINO
001 – PROJETO ABC	10/01/2013	
002 – PROJETO BCD	28/10/2012	10/03/2013
003 – PROJETO CDE	09/12/2012	
004 – PROJETO DEF	15/12/2012	

PROJETO	FUNCIONÁRIO	DATA ENTRADA	DATA SAÍDA
001 – PROJETO ABC	Antonia Camargo	10/01/2013	
002 – PROJETO BCD	Antonia Camargo	28/10/2012	10/03/2013
003 – PROJETO CDE	João da Silva	09/12/2012	
003 – PROJETO CDE	Joaquim Xavier	15/01/2013	

Figura 15 - Exemplo das informações atuais armazenadas em planilhas
Fonte: Elaborado pelo autor (2022)

Após o entendimento inicial, foram realizadas algumas reuniões em que os requisitos foram esclarecidos e o escopo (abrangência – trabalho que deve ser entregue) do projeto foi determinado, conforme segue:

A sprint 1 do projeto será composta pelo cadastro de funcionários e dependentes. Precisamos das informações dos dependentes para que possamos, em uma sprint futura, usá-las para o cálculo de proventos dos funcionários. Além de serem informações importantes para algumas iniciativas internas, como ações inovadoras que conciliam trabalho e família.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Na primeira sprint, também é necessário cadastrar todos os projetos desenvolvidos pela “TUDO Implanta”, não iniciados, iniciados ou finalizados.

E ainda é necessário saber, para todos os projetos implantados, os funcionários que realizaram atividades nestes projetos, bem como o período em que essas atividades foram desenvolvidas. Essas informações são importantes para que, em uma sprint futura, seja possível calcular o custo de implantação destes projetos.

Podemos ter funcionários de diferentes departamentos atuando na implantação dos projetos, e para o cálculo do custo, é importante sabermos qual departamento é mais oneroso.

Dados que precisam ser armazenados:

- **Departamento:** são divisões de uma empresa, onde cada divisão tem um propósito. Precisamos do código numérico, nome do departamento e uma sigla com três letras para facilitar a identificação do departamento em relatórios.
- **Funcionário:** é uma pessoa que tem skill para desenvolver as tarefas para as quais ele foi contratado. Precisamos do número da matrícula, nome completo do funcionário, data de nascimento, data da admissão, endereço completo (apenas residencial) e o salário mensal.
- **Dependente:** são as pessoas que possuem um grau de parentesco com o funcionário, por exemplo, filhos, esposa ou marido. Neste primeiro momento, precisamos do nome e data de nascimento de cada dependente do funcionário e um código para identificação de cada dependente.
- **Projeto:** somos uma empresa que desenvolve soluções na área de TI. Desenvolvemos projetos para todas as áreas da TI, bem como as diferentes etapas do processo de desenvolvimento de soluções. Precisamos do código do projeto numérico, nome ou título do projeto, a data em que o projeto foi iniciado e a data em que o projeto foi finalizado.
- **Implantação:** refere-se aos projetos desenvolvidos ou que estão em desenvolvimento, bem como os funcionários que estão desempenhando tarefas nesses projetos e em qual período. Precisamos de um código

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

para a implantação numérico, o projeto e o funcionário que participa dele e o período (entrada/saída) em que o funcionário atuou.

- **Regras de Negócio:** são as diretrizes que definem ou restringem ações, mostrando como as operações devem ser conduzidas e devem ser atendidas pela proposta de solução.

  - Um funcionário possui apenas um endereço, o endereço residencial.
  - Um funcionário possui apenas uma data de admissão.
  - Um funcionário possui um único número de matrícula.
  - Todos os funcionários possuem data de nascimento.
  - Cada dependente deve ter uma data de nascimento.
  - Não existem nomes de departamentos repetidos.
  - Não existem projetos com nomes repetidos.
  - Todos os funcionários possuem um salário acima ou igual ao salário-mínimo (em 2022, R$ 1.212,00). O maior salário atualmente é de R$ 18.000,00.
  - A data de término de um projeto deve ser maior que a data de início de um projeto.
  - A data de saída de um funcionário de um projeto deve ser maior que a data de entrada.
  - A data de término de um projeto será informada ao término do projeto em questão.
  - A data de saída de um funcionário de uma implantação será informada no momento da finalização das tarefas do funcionário no referido projeto.
  - Cada departamento pode alocar vários funcionários.
  - Cada funcionário deve pertencer a um único departamento.
  - Cada funcionário pode ter um ou mais dependentes (filhos, esposa ou marido).
  - Cada dependente deve pertencer a um único funcionário.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

- Não será considerada a hipótese de termos um casal trabalhando na mesma empresa (teriam dependentes comuns).
- Cada projeto pode ter várias implantações realizadas por funcionários.
- Podem existir projetos onde a implantação ainda não foi iniciada.
- Cada funcionário pode participar de várias implantações de projetos.
- Cada implantação deve ser realizada por único funcionário.
- Cada implantação refere-se a um período em que um funcionário atuou em determinado projeto.
- Podem existir funcionários que não participaram de nenhuma implantação.
- Cada funcionário pode participar da implantação de um mesmo projeto várias vezes, ou seja, um funcionário pode participar da implantação de um projeto em diferentes momentos da implantação.
- Um departamento é identificado por um código numérico de três dígitos.
- Um funcionário é identificado pelo número de matrícula, com cinco dígitos.
- Um dependente é identificado pelo código numérico de até dois dígitos mais o número de matrícula do funcionário que ele depende.
- Um projeto é identificado por um código numérico de cinco dígitos.
- Uma implantação é identificada pelo código da implantação, numérico com até três dígitos, mais o código do projeto que está implantação pertence.

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

1.1.6.2 Proposta de solução para a representação do modelo relacional (físico) de dados a partir da ferramenta Oracle Data Modeler

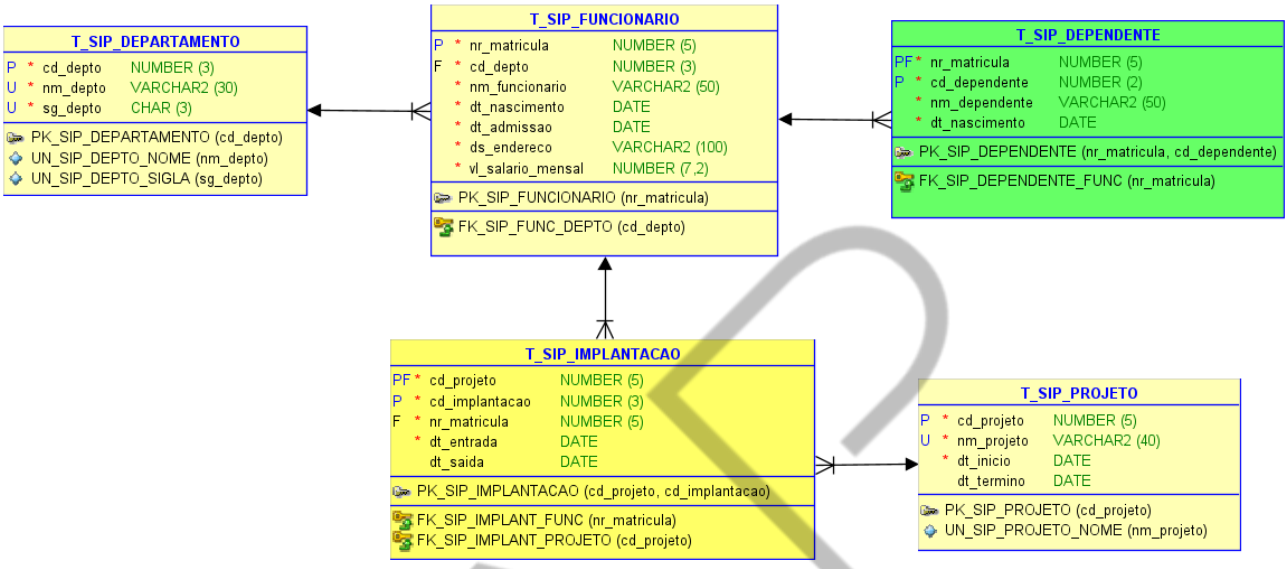

Figura 16 - Modelo Relacional (Físico)
Fonte: Elaborado pelo autor (2022)

1.1.6.3 Comandos da divisão DDL, da linguagem SQL para implementar o banco de dados referente ao estudo de caso SIP

Os scripts de banco de dados são arquivos que contêm instruções da linguagem SQL, que são utilizadas para realizar a implementação de um banco de dados em um SGBDR. Os scripts fazem parte da etapa de implantação (pré e pós-implantação) de banco de dados, compondo o projeto de banco de dados.

Todos os comandos desenvolvidos como exemplos nas seções anteriores devem ser inseridos em um arquivo e executados, utilizando-se de uma ferramenta que interprete e execute os comandos.

Como sugestão, podemos utilizar a ferramenta SQL Developer da Oracle.

O do SCRIPT (arquivo com comandos SQL) deve possuir um nome que ajude na sua identificação, por exemplo: “SCRIPT_DDL_PROJETO_SIP.SQL”, com a extensão SQL.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
-- SCRIPT PARA CRIAÇÃO DO BANCO DE DADOS

-- DO ESTUDO DE CASO SIP

-- COMANDOS PARA EXCLUIR AS TABELAS

DROP TABLE t_sip_departamento CASCADE CONSTRAINTS;
DROP TABLE t_sip_dependente CASCADE CONSTRAINTS;
DROP TABLE t_sip_funcionario CASCADE CONSTRAINTS;
DROP TABLE t_sip_implantacao CASCADE CONSTRAINTS;
DROP TABLE t_sip_projeto CASCADE CONSTRAINTS;

-- COMANDOS PARA CRIAR AS TABELAS

-- E ADICIONAR CONSTRAINTS: PRIMARY KEY, CHECK E UNIQUE

CREATE TABLE t_sip_departamento (
    cd_depto NUMBER(3)       NOT NULL,
    nm_depto VARCHAR2(30)    NOT NULL,
    sg_depto CHAR(3)         NOT NULL
);

COMMENT ON COLUMN t_sip_departamento.cd_depto IS
    'Um departamento é identificado por um código numérico de
três dígitos.';

   ALTER TABLE t_sip_departamento
ADD CONSTRAINT pk_sip_departamento
   PRIMARY KEY ( cd_depto );

   ALTER TABLE t_sip_departamento
ADD CONSTRAINT un_sip_depto_nome
        UNIQUE ( nm_depto );

   ALTER TABLE t_sip_departamento
ADD CONSTRAINT un_sip_depto_sigla
        UNIQUE ( sg_depto );

CREATE TABLE t_sip_dependente (
    nr_matricula  NUMBER(5)      NOT NULL,
    cd_dependente NUMBER(2)      NOT NULL,
    nm_dependente VARCHAR2(50)   NOT NULL,
    dt_nascimento DATE           NOT NULL
);

   ALTER TABLE t_sip_dependente
ADD CONSTRAINT pk_sip_dependente
   PRIMARY KEY ( nr_matricula, cd_dependente );
```

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
CREATE TABLE t_sip_funcionario (
    nr_matricula      NUMBER(5)      NOT NULL,
    cd_depto          NUMBER(3)      NOT NULL,
    nm_funcionario    VARCHAR2(50)   NOT NULL,
    dt_nascimento     DATE           NOT NULL,
    dt_admissao       DATE           NOT NULL,
    ds_endereco       VARCHAR2(100)  NOT NULL,
    vl_salario_mensal NUMBER(7, 2) NOT NULL
);

   ALTER TABLE t_sip_funcionario
ADD CONSTRAINT ck_sip_func_salario
         CHECK ( vl_salario_mensal >= 1212 );

   ALTER TABLE t_sip_funcionario
ADD CONSTRAINT pk_sip_funcionario
   PRIMARY KEY ( nr_matricula );

CREATE TABLE t_sip_implantacao (
    cd_projeto     NUMBER(5) NOT NULL,
    cd_implantacao NUMBER(3) NOT NULL,
    nr_matricula   NUMBER(5) NOT NULL,
    dt_entrada     DATE      NOT NULL,
    dt_saida       DATE          NULL
);

   ALTER TABLE t_sip_implantacao
ADD CONSTRAINT ck_sip_implant_data
         CHECK ( dt_saida > dt_entrada );

   ALTER TABLE t_sip_implantacao
ADD CONSTRAINT pk_sip_implantacao
   PRIMARY KEY ( cd_projeto, cd_implantacao );

CREATE TABLE t_sip_projeto (
    cd_projeto NUMBER(5)    NOT NULL,
    nm_projeto VARCHAR2(40) NOT NULL,
    dt_inicio  DATE         NOT NULL,
    dt_termino DATE
);

-- ADICIONAR CONSTRAINT FOREIGN KEY

   ALTER TABLE t_sip_projeto
ADD CONSTRAINT ck_sip_projeto_data
         CHECK ( dt_termino > dt_inicio );

   ALTER TABLE t_sip_projeto
ADD CONSTRAINT pk_sip_projeto
```

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
   PRIMARY KEY ( cd_projeto );

   ALTER TABLE t_sip_projeto
ADD CONSTRAINT un_sip_projeto_nome
        UNIQUE ( nm_projeto );

   ALTER TABLE t_sip_dependente
ADD CONSTRAINT fk_sip_dependente_func
   FOREIGN KEY ( nr_matricula )
    REFERENCES t_sip_funcionario ( nr_matricula );

   ALTER TABLE t_sip_funcionario
ADD CONSTRAINT fk_sip_func_depto
   FOREIGN KEY ( cd_depto )
    REFERENCES t_sip_departamento ( cd_depto );

   ALTER TABLE t_sip_implantacao
ADD CONSTRAINT fk_sip_implant_func
   FOREIGN KEY ( nr_matricula )
    REFERENCES t_sip_funcionario ( nr_matricula );

   ALTER TABLE t_sip_implantacao
ADD CONSTRAINT fk_sip_implant_projeto
   FOREIGN KEY ( cd_projeto )
    REFERENCES t_sip_projeto ( cd_projeto );
```

Código-fonte 17 - Script de criação do banco de dados
Fonte: Elaborado pelo autor (2022)

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

REFERÊNCIAS

ELMASRI, R.; NAVATHE, S.B. **Sistemas de Banco de Dados**: Fundamentos e Aplicações. 6.ed. São Paulo: Pearson Addison Wesley, 2011.

HEUSER, C.A. **Projeto de Banco de Dados. Série Livros Didáticos**. v. 4. 6.ed. Rio Grande do Sul: Bookman, 2009.

MACHADO, Felipe Nery R. **Banco de Dados - Projeto e Implementação**. 2.ed. São Paulo: Érica, 2002.

PRICE, J. **Oracle Database 11g SQL**. São Paulo: Bookman, 2009.

PUGA, S.; FRANÇA, E.; GOYA, M. **Banco de Dados**: Implementação em SQL, PL/SQL e Oracle 11g. São Paulo: Pearson Education do Brasil, 2013.

SILBERSCHATZ, A; KORTH, H. F.; SUDARSHAN, S. **Sistema de Banco de Dados**. 5.ed. Rio de Janeiro: Campus, 2006.

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com